Group project — the team document

ITWS 2110 · Fall 2026 ·

This is the one document a team works from. It has five parts: how the repository works, the pitch, the midterm, the final presentation and the final report. Parts 2, 3 and 5 are templates: you fill them in. The project brief in the course repository (`itws2110-course/project`) says what to build and why; this document says what to hand in, when, and how it is judged.

Section: 0_ · **Team:** <number> · **Members:** <names, RCS ids>

Contents

1. Overview — the deliverables, the repository layout, the rules that apply every time
2. Pitch — Fri 9/25 (done; kept here as the record)
3. Midterm: front-end prototype and API design — Fri 10/30
4. Final presentation and demo — Fri 12/11
5. Final report — Wed 12/16

Part 1 — Overview

The four deliverables

Deliverable	Due	In lab	In the repository	Share of project
Pitch (Part 2)	Fri 9/25	5 minutes	Part 2 of this document, or `PITCH.pptx`	15%
Midterm (Part 3)	Fri 10/30	8 minutes	The prototype in `web/`; Part 3 filled in	20%
Final presentation (Part 4)	Fri 12/11	12 minutes	The `v1.0` tag; the deployed app	30%
Final report (Part 5)	Wed 12/16, 11:59 PM	—	Part 5 filled in (or `REPORT.md`)	35%

The project is 25% of the course grade. Each deliverable gets one team grade. Individual grades are adjusted at the end; see Peer review and individual contributions below.

What every project has by the end

The same floor for every team. Reaching it, tested and deployed, is a B; what you build beyond it and how well you can explain it is the rest. The brief has the full table. In one line each:

- **Users.** Register, log in, log out. Two roles: user and admin.
- **One core resource**, in one table, belonging to a user.
- **A JSON API** for it: five endpoints (list, show, create, update, delete).
- **A React front end** that uses the API, styled with Tailwind.
- **Docker.** `docker compose watch` brings up everything on a clean clone.

- **Tests at three levels**, each its own Compose service.
- **Deployed** at a public URL from `main`.

Who may do what

This table is used in the pitch, the API design, the tests and the final demo. Decide it once.

Action	Anyone	Logged-in user	The owner	Admin
List, show	yes	yes	yes	yes
Create	no — `401`	yes	—	yes
Update	no — `401`	no — `403`	yes	yes
Delete	no — `401`	no — `403`	no — `403`	yes

Delete is admin-only in the minimum, including for the owner. If your application needs owners to delete their own items, say so in Part 3 and be ready to explain what happens to anything that pointed at the deleted item.

The repository

One repository per team. `main` is the product. Work on branches and merge by pull request, with at least one review. Everyone commits.

```
docker-compose.yml     one file at the root: web, api, db, and the test services
.env.example           every setting the system needs, with safe placeholder values
web/                   the React front end (Vite, Tailwind)
api/                   the Laravel API                          from week 9
PROJECT.docx           this document
```

Three commands, the same from the first commit to the last:

```
cp .env.example .env
docker compose watch                # the whole system
docker compose run --rm e2e         # one test suite; also: unit, api
```

Rules that apply to every deliverable

- **It runs in Docker.** A grader clones the repository, copies `.env.example` to `.env`, runs one command, and has the system. Nothing is installed on the laptop but Docker. If it only works on one machine, it is not handed in.
- **A feature without a test at the right level is not done.** `unit` (PHPUnit): the rules. `api` (Playwright request): every endpoint, success and each error. `e2e` (Playwright browser): the two key pages as a person uses them.
- **The course stack.** React, Tailwind, Laravel, MySQL, Docker Compose, PHPUnit, Playwright.

Peer review and individual contributions

There is no weekly log. Individual contribution is assessed once, at the end, from three sources:

- **A confidential peer review.** Each member completes a short form about every teammate, including themselves, after the final presentation. It is due with the final report, **Wed 12/16, 11:59 PM**. Only the instructor sees it. It asks what each person owned, how reliably they delivered, and how the team would have fared without them.
- **The repository.** Commits and pull requests by each member, and the reviews they gave. Everyone commits.
- **The presentations and the report.** Who answered which questions in lab, and each member's individual statement in the report.

Each deliverable gets one team grade. At the end, a member's project grade may be adjusted up or down from the team's grade based on these three together. One source alone does not decide it: a low peer rating with a strong commit history, or the reverse, leads to a conversation, not an automatic change.

If a teammate has stopped contributing, tell the instructor when it happens. The peer review at the end is too late to fix it.

Part 2 — Pitch

Due **Fri 9/25, 11:59 PM**, after the five-minute pitch in lab. Answer here, or in `PITCH.pptx` covering the same five questions. Every answer is a sentence or two. If an answer needs a paragraph, the scope is too big.

1. Who is it for?

One of: current students, future students, alumni. Then one specific kind of person in that group, and the thing they are trying to do.

2. What problem does it solve?

One sentence: what is broken or missing for that person today, and how your app fixes it. Then name the one thing you will store to solve it — the core resource — and list its fields, five to ten. Each record belongs to a user.

3. The five API endpoints

	Method and path	Who may call it
List	`GET /api/…`	anyone
Show	`GET /api/…/{id}`	anyone
Create	`POST /api/…`	logged in
Update	`PUT /api/…/{id}`	the owner, or an admin
Delete	`DELETE /api/…/{id}`	an admin

4. The two pages that matter most

Which two screens would a person use every time? These are the two pages you prototype for the midterm, and the two the browser tests cover.

Part 3 — Midterm: front-end prototype and API design

Due **Fri 10/30**: presented in lab, and in the repository by 11:59 PM. Two deliverables, built together and graded separately, 10% of the project each. No back end is expected. The point is to finish the front end and the contract first, so the Laravel API has something exact to implement.

What is due — a checklist

- `docker compose watch` serves the prototype from `web/`.
- The two key pages from the pitch, with client-side routing between them.
- At least one working form.
- All data comes from one module of fake data, shaped like the responses in this part.
- Sections A to C below, filled in.

The prototype

Requirement	What it means	How it is checked
Two key pages	The two from pitch question 4.	We click through both in lab
Navigation	A person can get from one page to the other and back without typing an address. React Router is the usual way.	In lab
One form	Creates or edits the core resource.	In lab
One data module	Every read and write goes through one file (for example `web/src/api.js`). In week 12 that file changes to call the real API and no page changes.	We open the file

A. The resource

<name> — one row per <thing>. Belongs to a user.

Field	Type	Required	Rules and notes
`id`	integer	—	assigned by the database
`user_id`	integer	—	the owner; set from the session, never from the request
…			for each of your fields: the rule in words ("required, at most 120 characters")
`created_at`, `updated_at`	timestamp	—	assigned

B. The five endpoints

Base path: `/api`. Fill in the path, the request body and the response for each. Keep the status codes.

Endpoint	Who	Request	Success	Errors
`GET /api/<things>`	anyone	—	`200` a list of items	—
`GET /api/<things>/{id}`	anyone	—	`200` the item	`404`
`POST /api/<things>`	logged in	the required fields	`201` the created item	`401`, `422`
`PUT /api/<things>/{id}`	owner or admin	any of the editable fields	`200` the updated item	`401`, `403`, `404`, `422`

DELETE /api/<things>/{id}	admin	—	204, no body	401, 403, 404

C. Errors

Status	When	Body
401	Nobody is logged in	`{ "message": "Unauthenticated." }`
403	Logged in, but not allowed to do this	`{ "message": "…" }`
404	No such item	`{ "message": "…" }`
422	The input broke a rule	`{ "message": "…", "errors": { "<field>": ["<what is wrong>"] } }`

In lab: eight minutes

- **The prototype.** `docker compose watch`, then both pages and the form.
- **The API design.** Sections A to C on screen. Expect questions on the choices, not the shapes: why is that field required, why is that endpoint admin-only, what happens to a user's items when the user is deleted.

How it is graded

Half	Full marks when
Prototype (10%)	It runs with one command. Both pages work, with routing between them. The form works. All data comes through one module.
API design (10%)	Every field has a type and a rule. All five endpoints have a route, who may call them, and their errors. The team can explain each choice.

Part 4 — Final presentation and demo

Fri 12/11, in lab. Every member presents part of it.

What to show

Present the problem, the audience and your key decisions. Then demo the two key pages running live against the Laravel API, brought up with `docker compose watch`. Expect questions like those in the midterm review.

Part 5 — Final report

Due in the repository by **Wed 12/16, 11:59 PM**.

Length: sections 1 to 7 together are 5-7 pages; diagrams and screenshots count, padding does not. Sections 8 to 10 are extra and do not count toward the length.

Members and roles: <name — what they owned>

1. The problem and the audience

Who it is for — the specific kind of current student, future student or alumnus — and what they were trying to do before your app existed. What you based that on: people you asked, your own experience, something you observed. Then what changed from the pitch, and why.

2. The system

- **Architecture diagram.** One picture: the browser, the React app, the Laravel API and MySQL, and where each runs when deployed.
- **Data model.** The tables, their fields, the relationships. The core resource and how it belongs to a user; anything added beyond the minimum.
- **The API as built.** The endpoints, and each place it differs from the midterm design, with the reason.
- **Running it.** The one command that brings the system up on a clean clone, the one that runs all three test suites, what is in `.env.example`, and what a grader sees who does exactly that and nothing else.

3. Decisions

Three or four decisions you made on purpose. For each: what you chose, the alternative you did not take, and why. The questions from the midterm review belong here: why that field is required, why that endpoint is admin-only, what happens to a user's items when the user is deleted.

4. Beyond the minimum

What things were added beyond the minimum requirements?

5. Testing

Suite	Tool	What it covers	What it leaves out, and why	Tests	Time
`unit`	PHPUnit				
`api`	Playwright request				
`e2e`	Playwright browser				

Then: one bug a test caught before a person did. Name the test.

6. Security

What you checked against the OWASP Top 10 and what you found. How credentials are handled.

7. Deployment

How a deploy happens from `main`, and how you would roll one back.

8. Process (does not count toward the length)

How the team worked: how work was split, how you met, how pull requests were reviewed. The three worst moments — what happened, how you found out, what you changed.

9. Individual statements (does not count toward the length)

Each member, one paragraph, in their own words: what they built, what they learned, one thing a teammate did that made the project better. Name at the end. The confidential peer review is separate from this and is submitted on its own.

10. Advice to next year's teams (does not count toward the length)

Five bullets. The kind you wish you had read in September.

AI Use Statement (required, one per team): which tools, what for, what you changed.